

Basic (design on paper before touching code) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `architecture_comparison_practice.py` (**Basic section**): a prediction written *before* any code is the only way to find out later whether your design instinct was right. This section is plain text, kept as a comment block at the top of the file, so the measured numbers below it can be checked against it. **If not:** after building, you'd remember "I expected that" — whether you did or not.

Task chosen for both depths below: "research the top 3 causes of X, then write a 3-sentence summary." Read both depths — they're not "wrong, right," they're 2 real, valid levels of the same design write-up, with real differences in how much you can defend at each one.

Basic Version

Approach 1 — one paragraph and a gut guess per design

Story: a quick first guess for each design, before thinking too hard — the honest starting point you'll compare against. **If not:** you'd skip straight to building, and never learn how good your instincts are.

```
design 1 -- single agent
    one agent, has a research tool, writes the summary itself in the
    same turn
    guess: cost low (1-2 calls), speed fastest, risk = one prompt doing
    two jobs at once

design 2 -- sequential
    research_agent runs first, hands its output straight to write_agent
    guess: cost medium (2 calls), speed about the same as design 1,
    risk = bad research passes straight through to a bad summary,
    nothing catches it

design 3 -- supervisor
    a router agent decides to call research_agent, then write_agent
    guess: cost highest (routing + 2 calls), speed slowest, risk =
    everything design 2 has, plus the router could pick the wrong
    specialist or call one twice
```

This is a genuinely useful first pass — it names the right shape for all three designs and gets the cost/speed ranking right. It's missing a specific "if this goes wrong" line for each design, and it doesn't yet count the supervisor's routing call as its own real cost rather than folding it into "the work."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the same designs, written the way you'd defend them out loud

Story: a guess becomes useful once it has numbers and mechanisms a colleague could argue with.

If not: "the supervisor is slower" can't be proven wrong — so it can't teach you anything either.

```
design 1 -- single agent
  flow: one agent gets "research X, write a summary," calls a research tool as
        needed inside its own turn, then writes the final summary itself
  cost: ~1-2 LLM calls total
  speed: fastest of the three -- no agent-to-agent handoff at all
  reliability risk: the single system prompt has to be good at both research
        judgment and writing style simultaneously; a prompt tuned for good writing
        style can quietly get worse at judging source quality, and vice versa

design 2 -- sequential
  flow: research_agent(topic) -> findings; write_agent(findings) -> summary,
        always in that fixed order
  cost: ~2 LLM calls, one per stage
  speed: about the same wall-clock time as design 1 -- there's no parallelism
        to gain here, since writing genuinely needs the research first
  reliability risk: very predictable in structure (the order is enforced by
        code, not judgment), but there is no check step -- a weak research_agent
        output is faithfully summarized as if it were solid

design 3 -- supervisor
  flow: supervisor reads the task, decides to call research_agent, gets the
        result, decides to call write_agent, gets the result, returns
  cost: ~3+ LLM calls -- 2 for the actual work, 1 or more for the routing
        decision itself, which is not free
  speed: slowest of the three -- the routing decision adds real delay on
        top of design 2's delay, for a task whose order was fixed anyway
  reliability risk: everything design 2 has, plus a new failure mode --
        the supervisor could route to the wrong specialist, or call one specialist
        twice thinking it needs another pass
```

Difference from Basic: the flow, cost, speed, and risk lines are now written the way you'd actually say them to a colleague — specific numbers, specific mechanisms ("a prompt tuned for good writing style can quietly get worse at judging source quality"), not just a label. This version would survive a

first follow-up question. It still doesn't spell out what each design's failure actually *looks like* to someone watching it happen — that's what Approach 2 adds.

Approach 2 — with a failure-mode line and the routing cost counted honestly

Story: a design is judged as much by how it fails as by how it works, and a supervisor's routing call is real cost, not free glue. **If not:** you'd pick the design that looks best when everything goes right.

```
design 1 -- single agent
  cost: ~1-2 LLM calls -- cheapest, since there's no handoff at all
  speed: fastest of the three
  reliability risk: one prompt has to be good at both research judgment and
  writing style at once
  if it goes wrong: it writes a confident-sounding summary from thin or wrong
  research, with nothing external to catch it -- this is the quietest failure
  of the three, because it never looks broken

design 2 -- sequential
  cost: ~2 LLM calls, one per stage -- research_agent's call, then
  write_agent's
  speed: about the same as design 1, since there's no real parallelism
  to gain
  reliability risk: the order is enforced by code, not judgment, so
  it's very predictable in structure, but there's no check between
  the two stages
  if it goes wrong: research_agent's mistake propagates straight
  through -- at least the failure is visible in isolation (you can
  inspect research_agent's output on its own before blaming
  write_agent), unlike design 1

design 3 -- supervisor
  cost: ~3+ LLM calls -- 2 for the work, plus the routing decision
  itself, counted separately and honestly rather than folded into
  "the work" -- for a task this small, the routing call can cost
  nearly as much as one real stage
  speed: slowest of the three -- same work as design 2, plus routing overhead,
  for a task whose order never actually changes
  reliability risk: everything design 2 has, plus the supervisor could route
  to the wrong specialist or call one twice
  if it goes wrong: the supervisor could loop -- calling research_agent again
  thinking it still needs more, or sending write_agent an empty findings field
  it doesn't know how to handle -- this is the noisiest failure of the three,
  and also the easiest to notice from the routing_log alone
```

Why 4 agents and not 1, defended out loud: for *this specific task* — a fixed, two-step research-then-write job with no real branching — you can't actually defend more than 2 agents. There's no

moment where the right next step is genuinely unknown ahead of time, which is the one thing that would justify paying for a supervisor's routing call. If a skeptical senior engineer asked "why not just one agent," the honest answer for this task is: sequential is worth it only once you've actually watched the single-agent version struggle to be good at both jobs at once — and if that hasn't happened yet, single agent is still the right default.

Difference from Approach 1: each design now ends with a specific "if it goes wrong" sentence, describing what someone watching the run would actually observe — not just naming the risk category, but saying what it looks like on screen. The supervisor's routing cost is called out explicitly as comparable to a real work stage, not hand-waved as negligible. That's the difference between a prediction and a prediction you could defend under a follow-up question.

Which one should you actually write? For a real project, Approach 1's depth is the minimum bar before you write any code — it forces the cost/speed/reliability numbers into the open where you can actually be wrong about them, which is the entire point of doing this on paper first. Approach 2's "if it goes wrong" line is worth the extra few minutes every time, because it's usually the difference between noticing a design chose the wrong pattern and finding out three weeks later. For this specific task, the paper verdict is design 2 (sequential): the order never changes, so a supervisor's extra cost, delay, and routing-failure risk buys nothing. Save the supervisor pattern for tasks where the right specialist, or the right order, genuinely can't be decided ahead of time — that's the judgment this whole exercise trains, and it's the same judgment `sequential_measure` and `supervisor_compare` ask you to check against real numbers next.